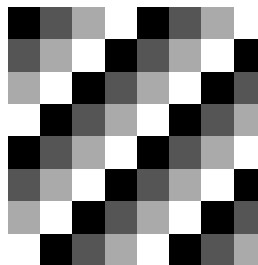


Shikha Nukala
Email ID: desigirl@umich.edu

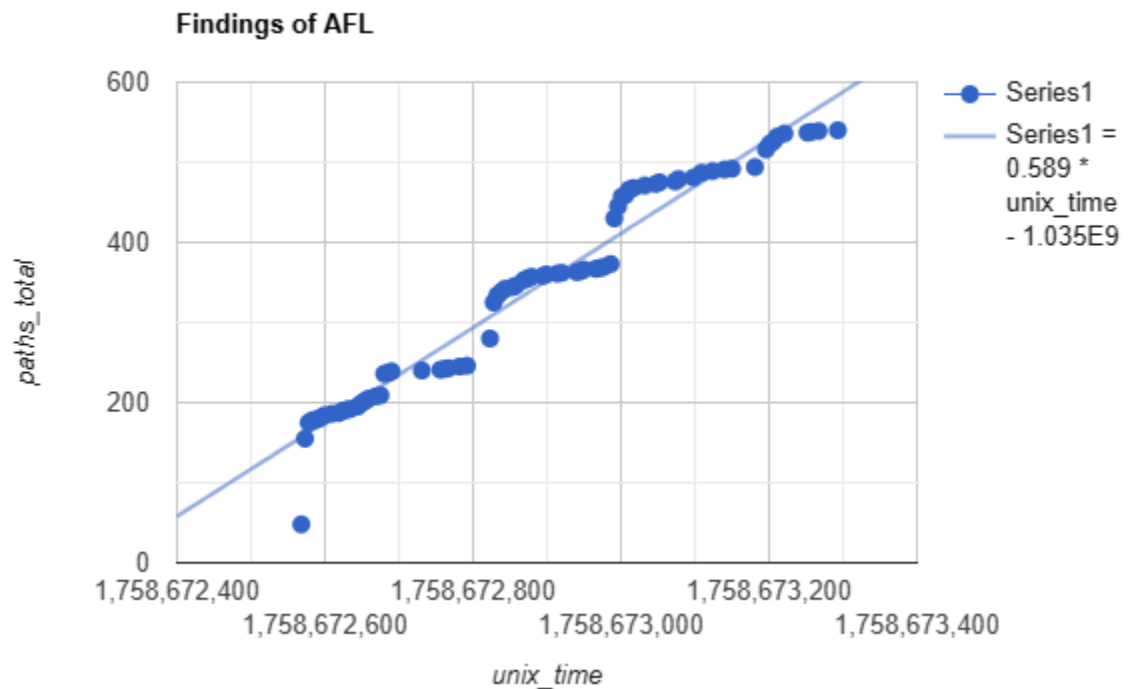
1. My AFL ran for 12 minutes and 17 seconds and created 540 test cases. It found some test paths that my manual tests from HW1 didn't cover. AFL mainly was different because it made corrupted PNG files that broke the libpng parser which is not something I tested for during HW 1 at all. This triggered error handling code which increased coverage. My manual tests had normal PNG files with different colors and sizes. These tested the main parts of libpng. AFL was good at finding crashes and my tests covered the rest of the normal image decoding. Using gcov to measure coverage on libpng, for seed images alone there was 18.7% line coverage and 14.2% branch coverage. For the AFL complete output with seeds and AFL generated images there was 20.4% line coverage and 16.8% branch coverage which means AFL added 2.7% more line coverage and 2.6% more branch coverage. AFL's output had higher coverage than just the seed images with a smaller improvement in the percentage. The 540 paths AFL found mostly hit error handling code that added some coverage but did not increase it drastically.

2.



This generated test file is particularly interesting because it represented a corruption in one of the images that I added that still maintains the image structure to be processed by libpng's parser but it likely was seen by error-handling code paths because of the slight corruption of the image compared to the added image I originally had which is a test case I did not think to cover.

3.



The scatterplot shows the AFL path discovery over time which I ran until it reached 540 total paths. The curve demonstrates AFL behavior with fast initial discovery of more than 200 paths in the first few minutes then less paths per unit of time as the more rare paths were discovered. The leveling off after approximately 500 seconds suggests AFL had gone through easily discoverable paths in the given timeframe, which is consistent with fuzzing literature that we read that says new discovery is exponentially slower.

4. Looking at the JaCoCo reports, both test suites covered the same amount of code which is 54% of instructions and 46% of branches. But the combined report is just a mix of the two test sets. It includes both the developer tests and the Randoop-generated tests. Since the numbers are the same for every package (like `org.jfree.chart.renderer.xy` at 32%, `org.jfree.chart.plot` at 67%, `org.jfree.chart.axis` at 55%), this means that Randoop's tests didn't really cover any new code that the developer tests didn't already cover. This probably means either Randoop made tests that were very similar to the developer tests, or it was set up to add to the existing tests without really increasing coverage. Overall,

Randoop didn't do any better or worse than the developer tests but didn't cover any extra code for my analysis.

5. Randoop did well in the `org.jfree.chart.axis` package, reaching 55% instruction coverage. It automatically generated tests for axis classes, trying different settings, boundary values, and unusual combinations like negative ranges or zero-width intervals. These tests caught errors and validation issues that the developer tests likely missed. The downside is that the tests are hard to read and don't represent real-world use cases, which could make debugging harder for the developer.
6. CRandoop performed poorly on the `org.jfree.chart.plot.flow` package, with only 18% instruction coverage. This is likely because these classes need specific setup or objects in a certain order. Randoop's random testing approach can't handle these dependencies, while human testers naturally understand how to set up the objects correctly. To ensure this is less likely, I would temporarily remove all developer-provided tests by modifying the test configuration to exclude the existing test classes, then run JaCoCo coverage analysis on only the Randoop generated tests.
7. AFL is strong at finding edge cases and error-handling paths by mutating inputs. It's great for security testing and finding crashes but needs good starting inputs and struggles with complex formats. Randoop is good for exploring APIs and achieving high coverage for individual classes, but its tests are hard to read and maintain. For a project, both tools can be useful but in different ways. Use AFL when testing parsers, file handlers, or programs where crashes and vulnerabilities are a concern. Use Randoop to supplement manual testing for library development or API testing, especially when you want broad coverage of methods and parameter validation. However it is not good to rely only on Randoop, since its tests can be confusing and hard to debug.